

Application Layer

The application layer is the top layer of the TCP/IP model. It provides network services directly to user applications such as browsers, email clients, chat apps, file transfer tools, and DNS resolvers.

In the TCP/IP model:

Layer	Examples
Application	HTTP, HTTPS, DNS, SMTP, FTP, WebSocket
Transport	TCP, UDP
Internet	IP, ICMP
Network Access	Ethernet, Wi-Fi

In the OSI model, the TCP/IP application layer roughly includes:

OSI Layer	Role
Application	User-facing network services
Presentation	Encoding, compression, encryption
Session	Session creation, management, termination

What The Application Layer Does

- Defines how applications communicate over a network.
- Defines message formats, commands, headers, status codes, and error handling.
- Uses transport layer protocols such as TCP or UDP.
- Provides services such as web browsing, email, file transfer, naming, authentication, and real-time communication.
- Often follows the client-server model, though peer-to-peer designs also exist.

Client-Server Model

In the client-server model:

- The client initiates communication.
- The server waits for requests and responds.
- The client usually has a temporary port.
- The server listens on a well-known port.

Example:

```
Browser  -> Web Server
Client   -> Server
```

Request -> Response

Common well-known ports:

Protocol	Port	Transport
HTTP	80	TCP
HTTPS	443	TCP
DNS	53	UDP/TCP
SMTP	25	TCP
SMTPS	465	TCP
SMTP submission	587	TCP
FTP control	21	TCP
FTP data	20	TCP
SSH/SFTP	22	TCP
POP3	110	TCP
POP3S	995	TCP
IMAP	143	TCP
IMAPS	993	TCP

HTTP

HTTP stands for HyperText Transfer Protocol. It is an application layer protocol used to transfer web resources such as HTML pages, CSS, JavaScript, images, videos, JSON, and files.

HTTP is:

- Client-server based.
- Request-response based.
- Stateless by default.
- Text-based in HTTP/1.1.
- Usually built on TCP.
- Used by browsers, APIs, mobile apps, CDNs, proxies, and microservices.

HTTP Request

A client sends an HTTP request to a server.

Basic format:

```
GET /index.html HTTP/1.1
Host: example.com
```

```
User-Agent: Mozilla/5.0
Accept: text/html
```

Parts of an HTTP request:

Part	Meaning
Method	Action to perform, such as GET or POST
URL/path	Resource being requested
Version	HTTP version
Headers	Metadata about the request
Body	Optional data sent to the server

HTTP Response

Basic format:

```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 1256

<html>...</html>
```

Parts of an HTTP response:

Part	Meaning
Version	HTTP version
Status code	Result of request
Reason phrase	Short text meaning
Headers	Metadata about response
Body	Actual content

Common HTTP Methods

Method	Purpose	Safe	Idempotent
GET	Retrieve a resource	Yes	Yes
POST	Submit data or create resource	No	No
PUT	Replace a resource completely	No	Yes
PATCH	Partially update a resource	No	Usually no

Method	Purpose	Safe	Idempotent
DELETE	Delete a resource	No	Yes
HEAD	Like GET, but without response body	Yes	Yes
OPTIONS	Ask server which methods/options are supported	Yes	Yes
TRACE	Diagnostic loopback method	Yes	Yes

Safe means the method should not modify server state.

Idempotent means repeating the same request multiple times has the same effect as sending it once.

GET vs POST

Feature	GET	POST
Purpose	Fetch data	Send data
Data location	URL query string	Request body
Visibility	Visible in URL	Not visible in URL
Bookmarkable	Yes	No
Cacheable	Usually yes	Usually no
Idempotent	Yes	No
Used for login?	No	Yes

Example GET:

```
GET /search?q=networking HTTP/1.1
```

Example POST:

```
POST /login HTTP/1.1
Content-Type: application/x-www-form-urlencoded

username=abc&password=xyz
```

Important HTTP Headers

Header	Purpose
Host	Domain name of the server
User-Agent	Client software information

Header	Purpose
Accept	Response formats accepted by client
Accept-Language	Preferred language
Content-Type	Type of request/response body
Content-Length	Size of body in bytes
Authorization	Credentials or token
Cookie	Cookies sent from client to server
Set-Cookie	Server asks client to store cookie
Cache-Control	Caching rules
ETag	Resource version identifier
Location	Redirection target
Referer	Previous page URL
Origin	Origin of the request
Access-Control-Allow-Origin	CORS permission
Connection	Connection behavior

HTTP Versions

Version	Main Features
HTTP/0.9	Very simple, only GET
HTTP/1.0	Headers, status codes, separate TCP connection per request
HTTP/1.1	Persistent connections, chunked transfer, Host header
HTTP/2	Binary framing, multiplexing, header compression
HTTP/3	Uses QUIC over UDP, faster connection setup, improved performance

HTTP/1.1

Important features:

- Persistent connections by default.
- Multiple requests can use the same TCP connection.
- Host header supports virtual hosting.
- Chunked transfer encoding allows streaming unknown-sized responses.

Problem:

- Head-of-line blocking can happen because requests on one TCP connection may wait behind others.

HTTP/2

Important features:

- Binary protocol instead of plain text.
- Multiplexing: many streams over one TCP connection.
- Header compression using HPACK.
- Server push was introduced, though it is now rarely used.

Benefit:

- Reduces latency compared with HTTP/1.1.

Limitation:

- Still uses TCP, so TCP-level head-of-line blocking can affect all streams if packet loss occurs.

HTTP/3

Important features:

- Runs over QUIC.
- QUIC runs over UDP.
- Built-in encryption using TLS 1.3 concepts.
- Faster connection setup.
- Better handling of packet loss.
- Connection migration, useful when switching from Wi-Fi to mobile data.

HTTP Is Stateless

HTTP is stateless, meaning each request is independent. The server does not automatically remember previous requests from the same client.

Because of this, web applications use:

- Cookies.
- Sessions.
- Tokens.
- Hidden form fields.
- URL parameters.

Example:

```
Request 1: GET /login  
Request 2: POST /login  
Request 3: GET /profile
```

HTTP itself does not remember that request 3 belongs to the logged-in user. The application uses cookies or tokens to identify the user.

HTTPS

HTTPS stands for HyperText Transfer Protocol Secure. It is HTTP over TLS.

HTTPS provides:

- Confidentiality: attackers cannot read the data.
- Integrity: attackers cannot modify data without detection.
- Authentication: client can verify the server identity using certificates.

Default port:

443

HTTP vs HTTPS

Feature	HTTP	HTTPS
Security	Not encrypted	Encrypted using TLS
Port	80	443
URL	http://	https://
Certificate	Not required	Required
Data protection	No confidentiality	Confidentiality and integrity
Safe for passwords	No	Yes

TLS Handshake Simplified

Typical TLS handshake idea:

1. Client sends supported TLS versions, cipher suites, and random value.
2. Server chooses TLS version and cipher suite.
3. Server sends certificate.
4. Client verifies certificate using trusted Certificate Authorities.
5. Client and server agree on shared keys.
6. Encrypted communication begins.

Certificate

A digital certificate proves the identity of a server.

It contains:

- Domain name.
- Public key.
- Certificate Authority signature.
- Validity period.

- Issuer information.

Certificate Authority

A Certificate Authority is a trusted organization that signs certificates.

Examples:

- Let's Encrypt.
- DigiCert.
- GlobalSign.
- Sectigo.

Why HTTPS Matters

HTTPS prevents:

- Password sniffing.
- Session hijacking by passive sniffing.
- Man-in-the-middle modification.
- Tampering with web pages.
- Leakage of sensitive API data.

HTTPS does not automatically prevent:

- Phishing.
- Malware.
- Weak passwords.
- XSS.
- SQL injection.
- Server-side bugs.

REST

REST stands for Representational State Transfer. It is an architectural style for designing networked APIs, especially web APIs over HTTP.

REST is not a protocol. It is a set of design constraints.

REST Principles

Principle	Meaning
Client-server	Client and server are separated
Stateless	Each request contains all information needed
Cacheable	Responses can declare whether they can be cached
Uniform interface	Resources are accessed in a consistent way
Layered system	Proxies, gateways, and load balancers may exist between client and server

Principle	Meaning
Code on demand	Optional ability to send executable code

Resource

In REST, everything is treated as a resource.

Examples:

```
/users
/users/10
/orders
/orders/123/items
```

REST Uses HTTP Methods

Operation	HTTP Method	Example
Get all users	GET	GET /users
Get one user	GET	GET /users/10
Create user	POST	POST /users
Replace user	PUT	PUT /users/10
Update part of user	PATCH	PATCH /users/10
Delete user	DELETE	DELETE /users/10

REST Response Example

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "id": 10,
  "name": "Asha",
  "role": "developer"
}
```

REST Best Practices

- Use nouns in URLs, not verbs.
- Use HTTP methods for actions.
- Use plural resource names.
- Use status codes correctly.
- Use JSON consistently.

- Version APIs when needed.
- Support pagination for large collections.
- Validate input.
- Return clear error messages.
- Use HTTPS.

Good:

```
GET /users/10
POST /users
DELETE /users/10
```

Less ideal:

```
GET /getUser?id=10
POST /deleteUser
```

REST vs SOAP

Feature	REST	SOAP
Type	Architectural style	Protocol
Data format	Usually JSON	XML
Simplicity	Simpler	More strict
Transport	Usually HTTP	HTTP, SMTP, others
Performance	Usually lighter	Heavier
Use cases	Web/mobile APIs	Enterprise systems, banking, legacy systems

REST vs GraphQL

Feature	REST	GraphQL
Endpoint style	Multiple endpoints	Usually single endpoint
Data fetching	Server decides response shape	Client asks exact fields
Over-fetching	Possible	Reduced
Under-fetching	Possible	Reduced
Caching	Uses HTTP caching naturally	More custom
Learning curve	Lower	Higher

WebSocket

WebSocket is an application layer protocol that provides full-duplex communication over a single long-lived TCP connection.

Full-duplex means both client and server can send data at the same time.

WebSocket is useful for real-time applications.

Examples:

- Chat applications.
- Online games.
- Live stock prices.
- Collaborative editors.
- Notifications.
- Real-time dashboards.
- Multiplayer apps.

Why WebSocket Is Needed

HTTP is request-response based:

```
Client asks -> Server responds
```

The server cannot normally send data unless the client requests first.

WebSocket allows:

```
Client <-> Server
```

Both sides can send messages whenever needed.

WebSocket Handshake

WebSocket starts as an HTTP request and then upgrades the connection.

Example:

```
GET /chat HTTP/1.1
Host: example.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: abc123==
Sec-WebSocket-Version: 13
```

Server response:

```
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: ...
```

After this, the connection becomes a WebSocket connection.

WebSocket Ports

Scheme	Meaning	Port
ws://	WebSocket without TLS	80
wss://	WebSocket over TLS	443

WebSocket vs HTTP

Feature	HTTP	WebSocket
Communication	Request-response	Full-duplex
Connection	Often short or reused	Long-lived
Server push	Limited	Native
Overhead	Headers per request	Lower after handshake
Best for	Pages, APIs, resources	Real-time updates

WebSocket vs Polling

Polling:

```
Client asks every few seconds: Any update?
```

Long polling:

```
Client asks. Server waits until update exists, then responds.
```

WebSocket:

```
Server sends update immediately over open connection.
```

Technique	Latency	Overhead	Complexity
Polling	Higher	Higher	Low
Long polling	Medium	Medium	Medium
WebSocket	Low	Low after setup	Medium

DNS

DNS stands for Domain Name System. It translates human-readable domain names into IP addresses.

Example:

```
www.google.com -> 142.250.x.x
```

DNS is often called the phonebook of the internet.

Default port:

```
53
```

Transport:

- UDP for most queries.
- TCP for large responses and zone transfers.

Why DNS Is Needed

Humans remember names better than IP addresses.

Instead of:

```
142.250.183.4
```

we use:

```
google.com
```

DNS Hierarchy

DNS is hierarchical:

```
Root
|
+-- Top-Level Domain
    |
    +-- Second-Level Domain
        |
        +-- Subdomain
```

Example:

```
www.example.com
```

Part	Meaning
.	Root
com	Top-Level Domain
example	Second-Level Domain
www	Subdomain/host

DNS Resolution Process

When a browser wants the IP for `www.example.com`:

1. Browser checks its own cache.
2. Operating system checks local DNS cache.
3. Request goes to recursive resolver, often ISP or public DNS.
4. Recursive resolver asks root server.
5. Root server points to TLD server for `.com`.
6. TLD server points to authoritative server for `example.com`.
7. Authoritative server returns IP address.
8. Resolver returns IP to client.
9. Browser connects to the IP.

Recursive vs Iterative DNS Query

Query Type	Meaning
Recursive	Server must return final answer or error
Iterative	Server returns best next server to ask

Clients usually send recursive queries to recursive resolvers.

Resolvers use iterative queries to contact root, TLD, and authoritative servers.

DNS Servers

Server Type	Role
Root server	Points to TLD servers
TLD server	Points to authoritative servers
Authoritative server	Stores actual DNS records for domain
Recursive resolver	Performs lookup on behalf of client

Common DNS Record Types

Record	Purpose
A	Maps domain to IPv4 address
AAAA	Maps domain to IPv6 address
CNAME	Alias from one name to another
MX	Mail server for domain
NS	Name server for domain
TXT	Text data, often verification/security
SOA	Start of authority record
PTR	Reverse DNS lookup
SRV	Service location
CAA	Which CAs may issue certificates

DNS Caching

DNS uses caching to improve speed and reduce load.

TTL stands for Time To Live. It tells how long a DNS record may be cached.

High TTL:

- Fewer DNS lookups.
- Better performance.
- Slower changes during migration.

Low TTL:

- Faster DNS changes.
- More DNS traffic.

DNS Over UDP vs TCP

DNS commonly uses UDP because:

- Lower overhead.

- Faster for small queries.
- No connection setup.

DNS uses TCP when:

- Response is too large.
- Zone transfer is needed.
- Reliability is required.

DNS Security Problems

Common DNS attacks:

- DNS spoofing.
- DNS cache poisoning.
- DNS amplification DDoS.
- Domain hijacking.
- Typosquatting.

Protection:

- DNSSEC.
- DNS over HTTPS.
- DNS over TLS.
- Strong registrar security.
- Short-lived credentials and proper monitoring.

DNSSEC

DNSSEC adds digital signatures to DNS data.

It provides:

- Authentication of DNS data.
- Integrity of DNS data.

It does not provide:

- Encryption.
- Confidentiality.

SMTP

SMTP stands for Simple Mail Transfer Protocol. It is used to send email.

Default ports:

Port	Purpose
25	Server-to-server SMTP transfer
587	Mail submission by authenticated clients

Port	Purpose
465	SMTP over TLS, commonly used for secure submission

SMTP usually uses TCP.

Email Protocols

Protocol	Purpose
SMTP	Sending email
POP3	Downloading email from server
IMAP	Accessing and synchronizing email on server

SMTP sends mail. POP3 and IMAP retrieve mail.

SMTP Flow

```
Sender mail client
  -> Sender mail server
  -> Recipient mail server
  -> Recipient mail client
```

Example:

```
alice@gmail.com sends mail to bob@example.com
```

1. Alice's client submits mail to Gmail SMTP server.
2. Gmail checks DNS MX record for [example.com](#).
3. Gmail sends mail to recipient mail server.
4. Bob reads mail using IMAP or POP3.

SMTP Commands

Command	Purpose
HELO	Identify client
EHLO	Extended HELO
MAIL FROM	Sender address
RCPT TO	Recipient address
DATA	Start message body
RSET	Reset current transaction

Command	Purpose
VRFY	Verify mailbox
NOOP	No operation
QUIT	End session

Example SMTP conversation:

```
S: 220 mail.example.com SMTP Service Ready
C: EHLO client.com
S: 250 mail.example.com
C: MAIL FROM:<alice@example.com>
S: 250 OK
C: RCPT TO:<bob@example.org>
S: 250 OK
C: DATA
S: 354 End data with <CR><LF>.<CR><LF>
C: Subject: Hello
C:
C: Hi Bob
C: .
S: 250 Message accepted
C: QUIT
S: 221 Bye
```

SMTP Limitations

SMTP by itself:

- Does not retrieve email.
- Was not originally designed with strong authentication.
- Does not guarantee end-to-end encryption.
- Can be abused for spam if poorly configured.

Email Security Records

Mechanism	Purpose
SPF	Defines which servers may send email for a domain
DKIM	Adds digital signature to outgoing mail
DMARC	Policy for handling SPF/DKIM failures
MX	Defines mail servers for a domain

SMTP vs IMAP vs POP3

Feature	SMTP	IMAP	POP3
Main use	Send mail	Read/sync mail	Download mail
Direction	Client/server to server	Server to client	Server to client
Keeps mail on server	Not applicable	Yes	Usually downloads
Multi-device friendly	Not applicable	Yes	Less suitable
Default port	25	143	110
Secure port	465/587	993	995

FTP

FTP stands for File Transfer Protocol. It is used to transfer files between a client and a server.

Default ports:

Port	Purpose
21	Control connection
20	Data connection in active mode

FTP uses TCP.

FTP Connections

FTP uses two connections:

Connection	Purpose
Control connection	Commands and responses
Data connection	Actual file transfer

The control connection usually stays open during the session.

FTP Commands

Command	Purpose
USER	Username
PASS	Password
LIST	List files
RETR	Download file
STOR	Upload file
DELE	Delete file

Command	Purpose
CWD	Change directory
PWD	Print working directory
MKD	Make directory
RMD	Remove directory
QUIT	End session

FTP Active Mode

In active mode:

1. Client opens control connection to server port 21.
2. Client tells server which client port to connect to.
3. Server connects from port 20 to client data port.

Problem:

- Firewalls/NAT may block incoming connection to the client.

FTP Passive Mode

In passive mode:

1. Client opens control connection to server port 21.
2. Server tells client which server port to connect to for data.
3. Client opens data connection to that server port.

Benefit:

- Works better with NAT and firewalls.

FTP Security

Classic FTP is insecure because:

- Username and password are sent in plaintext.
- File data is sent in plaintext.
- Vulnerable to sniffing.

Secure alternatives:

Protocol	Meaning
FTPS	FTP over TLS
SFTP	SSH File Transfer Protocol, different from FTP
SCP	Secure Copy over SSH

FTP is now less common for modern secure file transfer.

Cookies

A cookie is a small piece of data stored by the browser for a website.

Cookies are sent automatically by the browser with matching requests.

They are commonly used for:

- Login sessions.
- User preferences.
- Tracking.
- Shopping carts.
- CSRF protection tokens.
- Personalization.

Set-Cookie Header

Server sends:

```
Set-Cookie: sessionId=abc123; HttpOnly; Secure; SameSite=Lax
```

Browser later sends:

```
Cookie: sessionId=abc123
```

Cookie Attributes

Attribute	Meaning
Expires	Absolute expiry date
Max-Age	Lifetime in seconds
Domain	Domain where cookie is valid
Path	URL path where cookie is valid
Secure	Sent only over HTTPS
HttpOnly	Not accessible from JavaScript
SameSite	Controls cross-site sending

SameSite Values

Value	Meaning
Strict	Cookie sent only for same-site requests
Lax	Sent for same-site and some top-level navigation

Value	Meaning
None	Sent cross-site, requires Secure

Session Cookies vs Persistent Cookies

Type	Meaning
Session cookie	Deleted when browser session ends
Persistent cookie	Stored until expiry time

First-Party vs Third-Party Cookies

Type	Meaning
First-party cookie	Set by the site the user is visiting
Third-party cookie	Set by another domain embedded in the page

Third-party cookies are often used for advertising and tracking. Modern browsers increasingly restrict them.

Cookie Security

Good practices:

- Use HTTPS.
- Set **Secure**.
- Set **HttpOnly** for session cookies.
- Set **SameSite=Lax** or **SameSite=Strict** where possible.
- Do not store passwords in cookies.
- Store random session IDs, not sensitive user data.
- Rotate session IDs after login.

Sessions

A session is server-side state associated with a client.

Because HTTP is stateless, sessions help applications remember users between requests.

Session Flow

1. User logs in.
2. Server verifies credentials.
3. Server creates a session record.
4. Server sends session ID in a cookie.
5. Browser sends session ID on later requests.
6. Server uses session ID to find the session data.

Example:

```
Cookie: sessionId=abc123
```

Server-side session store:

```
abc123 -> userId=42, role=admin, expires=...
```

Cookie vs Session

Feature	Cookie	Session
Stored where	Client browser	Server
Size	Small	Can be larger
Security	Can be modified by client unless protected	More secure
Lifetime	Controlled by expiry	Controlled by server
Used for	ID/preferences	Login state/user data

Session ID

A session ID should be:

- Random.
- Long enough.
- Unpredictable.
- Sent over HTTPS.
- Regenerated after login.
- Expired after inactivity.

Session Attacks

Attack	Meaning	Prevention
Session hijacking	Attacker steals session ID	HTTPS, HttpOnly, Secure
Session fixation	Attacker forces victim to use known session ID	Regenerate ID after login
CSRF	Browser sends authenticated request unknowingly	SameSite, CSRF tokens
XSS stealing cookie	JavaScript steals cookie	HttpOnly, XSS prevention

Token-Based Authentication

Modern APIs often use tokens.

Example:

```
Authorization: Bearer eyJhbGciOi...
```

Tokens may be:

- Opaque tokens.
- JWTs.

Sessions are usually server-side. Tokens are often self-contained or checked by an auth server.

HTTP Status Codes

HTTP status codes tell the result of a request.

Status code classes:

Class	Meaning
1xx	Informational
2xx	Success
3xx	Redirection
4xx	Client error
5xx	Server error

1xx Informational

Code	Meaning
100	Continue
101	Switching Protocols
102	Processing
103	Early Hints

Important:

- **101 Switching Protocols** is used in WebSocket upgrade.

2xx Success

Code	Meaning	Common Use
200	OK	Successful GET/POST
201	Created	Resource created
202	Accepted	Request accepted for async processing

Code	Meaning	Common Use
204	No Content	Success with no response body
206	Partial Content	Range request, video streaming/download resume

3xx Redirection

Code	Meaning	Common Use
301	Moved Permanently	Permanent URL change
302	Found	Temporary redirect
303	See Other	Redirect after POST
304	Not Modified	Cached version is still valid
307	Temporary Redirect	Temporary redirect, method preserved
308	Permanent Redirect	Permanent redirect, method preserved

301 vs 302

Code	Meaning
301	Permanent redirect
302	Temporary redirect

302 vs 307

Code	Difference
302	Some clients may change POST to GET
307	Method and body must be preserved

4xx Client Errors

Code	Meaning	Common Cause
400	Bad Request	Invalid syntax/input
401	Unauthorized	Authentication required or failed
403	Forbidden	Authenticated but not allowed
404	Not Found	Resource does not exist
405	Method Not Allowed	Method unsupported for resource
408	Request Timeout	Client took too long
409	Conflict	Resource state conflict

Code	Meaning	Common Cause
410	Gone	Resource permanently removed
413	Payload Too Large	Request body too large
414	URI Too Long	URL too long
415	Unsupported Media Type	Wrong Content-Type
418	I'm a teapot	Joke status code
422	Unprocessable Content	Validation failed
429	Too Many Requests	Rate limit exceeded
431	Request Header Fields Too Large	Headers too large

401 vs 403

Code	Meaning
401	Who are you? Authentication required
403	I know who you are, but you are not allowed

Example:

- Not logged in: **401 Unauthorized**.
- Logged in as normal user but accessing admin page: **403 Forbidden**.

5xx Server Errors

Code	Meaning	Common Cause
500	Internal Server Error	Generic server failure
501	Not Implemented	Server does not support feature
502	Bad Gateway	Invalid response from upstream server
503	Service Unavailable	Server overloaded/down for maintenance
504	Gateway Timeout	Upstream server did not respond in time
505	HTTP Version Not Supported	Unsupported HTTP version

502 vs 503 vs 504

Code	Meaning
502	Gateway received bad response from upstream
503	Server/service temporarily unavailable
504	Gateway waited too long for upstream

Caching In HTTP

HTTP caching improves performance by reusing previous responses.

Cache-Control

Examples:

```
Cache-Control: no-store  
Cache-Control: no-cache  
Cache-Control: max-age=3600  
Cache-Control: public, max-age=86400  
Cache-Control: private, max-age=600
```

Directive	Meaning
no-store	Do not store response
no-cache	Store but revalidate before reuse
max-age	Time response is fresh
public	Shared caches may store
private	Only browser cache may store

ETag

ETag is a version identifier for a resource.

Flow:

```
Server: ETag: "v1"  
Client next time: If-None-Match: "v1"  
Server: 304 Not Modified
```

Last-Modified

Flow:

```
Server: Last-Modified: Tue, 01 Jan 2025 10:00:00 GMT  
Client: If-Modified-Since: Tue, 01 Jan 2025 10:00:00 GMT  
Server: 304 Not Modified
```

CORS

CORS stands for Cross-Origin Resource Sharing.

Browsers enforce the Same-Origin Policy. A web page from one origin cannot freely read responses from another origin unless allowed by CORS.

Origin means:

```
scheme + host + port
```

Examples:

URL	Origin
https://example.com/page	https://example.com:443
http://example.com/page	http://example.com:80
https://api.example.com	https://api.example.com:443

These are different origins:

```
https://example.com
https://api.example.com
http://example.com
https://example.com:8443
```

Important CORS headers:

Header	Purpose
Origin	Sent by browser
Access-Control-Allow-Origin	Server allows origin
Access-Control-Allow-Methods	Allowed methods
Access-Control-Allow-Headers	Allowed headers
Access-Control-Allow-Credentials	Whether credentials are allowed

Preflight Request

For some cross-origin requests, browser sends an OPTIONS request first.

```
OPTIONS /api/users HTTP/1.1
Origin: https://app.example.com
Access-Control-Request-Method: POST
```

Server may respond:

```
Access-Control-Allow-Origin: https://app.example.com
Access-Control-Allow-Methods: POST
```

URL Structure

Example:

```
https://www.example.com:443/path/to/page?search=abc#section1
```

Part	Meaning
https	Scheme/protocol
www.example.com	Host/domain
443	Port
/path/to/page	Path
search=abc	Query string
section1	Fragment

MIME Types

MIME type tells the format of content.

Examples:

MIME Type	Meaning
text/html	HTML
text/css	CSS
application/javascript	JavaScript
application/json	JSON
image/png	PNG image
image/jpeg	JPEG image
multipart/form-data	File upload/form data
application/x-www-form-urlencoded	HTML form data

Proxies, Gateways, And Load Balancers

Proxy

A proxy acts on behalf of a client or server.

Forward proxy:

- Used by clients.
- Hides client from internet.
- Can filter traffic.
- Example: corporate proxy.

Reverse proxy:

- Sits in front of servers.
- Hides backend servers.
- Provides load balancing, TLS termination, caching, compression.
- Example: Nginx, HAProxy, cloud load balancer.

Load Balancer

A load balancer distributes requests across multiple servers.

Benefits:

- Better availability.
- Better performance.
- Scalability.
- Fault tolerance.

Algorithms:

- Round robin.
- Least connections.
- IP hash.
- Weighted round robin.

Gateway

A gateway connects different networks or protocols.

Example:

- API gateway routes requests to microservices.
- Email gateway filters mail.

Important Placement Questions

What happens when you type a URL in the browser?

1. Browser parses the URL.
2. Browser checks cache.
3. Browser performs DNS lookup if IP is not cached.
4. Browser opens TCP connection to server.
5. For HTTPS, TLS handshake happens.

6. Browser sends HTTP request.
7. Server processes request.
8. Server sends HTTP response.
9. Browser parses HTML.
10. Browser requests CSS, JavaScript, images, fonts, and other resources.
11. Browser builds DOM and CSSOM.
12. Browser renders page.

Why is HTTP stateless?

HTTP is stateless because each request is independent. The protocol does not require the server to remember previous requests. This makes HTTP simple and scalable, but applications need cookies, sessions, or tokens to remember users.

How does HTTPS protect data?

HTTPS uses TLS to encrypt communication between client and server. It also verifies server identity using certificates and protects data integrity using cryptographic checks.

Why does DNS usually use UDP?

DNS usually uses UDP because most DNS queries and responses are small, and UDP has lower overhead than TCP. DNS uses TCP for large responses, reliability, and zone transfers.

What is the difference between cookie and session?

A cookie is stored in the browser. A session is stored on the server. Usually, the cookie stores only the session ID, while actual user data is stored on the server.

What is REST?

REST is an architectural style for designing APIs. It treats data as resources and uses standard HTTP methods such as GET, POST, PUT, PATCH, and DELETE to operate on those resources.

What is WebSocket?

WebSocket is a protocol that enables full-duplex real-time communication between client and server over a long-lived TCP connection.

Why is FTP considered insecure?

Traditional FTP sends usernames, passwords, and data in plaintext. Secure alternatives include FTPS and SFTP.

What is the difference between 401 and 403?

401 Unauthorized means authentication is required or failed. **403 Forbidden** means the user is authenticated but does not have permission.

What is the difference between 500 and 503?

500 Internal Server Error means a generic server-side error occurred. **503 Service Unavailable** means the service is temporarily unavailable, often due to overload or maintenance.

Quick Revision Tables

Protocol Summary

Protocol	Full Form	Purpose	Port	Transport
HTTP	HyperText Transfer Protocol	Web communication	80	TCP
HTTPS	HTTP Secure	Secure web communication	443	TCP
DNS	Domain Name System	Name to IP mapping	53	UDP/TCP
SMTP	Simple Mail Transfer Protocol	Send email	25/587/465	TCP
FTP	File Transfer Protocol	Transfer files	21/20	TCP
WebSocket	WebSocket Protocol	Real-time full-duplex communication	80/443	TCP

Secure Alternatives

Insecure/Older	Secure Alternative
HTTP	HTTPS
FTP	FTPS, SFTP
Telnet	SSH
POP3	POP3S
IMAP	IMAPS
SMTP plaintext	SMTP with STARTTLS or SMTPS

Common Status Codes To Remember

Code	Meaning
200	OK
201	Created
204	No Content
301	Moved Permanently
302	Found
304	Not Modified

Code	Meaning
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
405	Method Not Allowed
409	Conflict
429	Too Many Requests
500	Internal Server Error
502	Bad Gateway
503	Service Unavailable
504	Gateway Timeout

Interview One-Liners

- HTTP is stateless, request-response based, and usually runs over TCP.
- HTTPS is HTTP over TLS and provides encryption, integrity, and authentication.
- DNS maps domain names to IP addresses and usually uses UDP port 53.
- SMTP sends email; IMAP and POP3 retrieve email.
- FTP uses separate control and data connections.
- REST is an architectural style, not a protocol.
- WebSocket starts with an HTTP upgrade and then provides full-duplex communication.
- Cookies are stored on the client; sessions are usually stored on the server.
- 401 means unauthenticated; 403 means unauthorized after authentication.
- 502, 503, and 504 often involve gateway or upstream service problems.

Common Mistakes To Avoid

- Saying REST is a protocol. It is an architectural style.
- Saying HTTPS is a different protocol from HTTP. It is HTTP over TLS.
- Saying DNS only uses UDP. DNS can also use TCP.
- Saying SMTP is used to read emails. SMTP sends emails.
- Saying cookies and sessions are the same. Cookies are client-side; sessions are server-side.
- Saying WebSocket is the same as HTTP polling. WebSocket uses a persistent full-duplex connection.
- Saying 401 means forbidden. 401 means authentication is needed or failed.
- Saying 403 means login is needed. 403 means access is denied even though identity may be known.